



Мікросервіси та бази даних

Проблеми з базою даних мікросервісів

101

Монолітний додаток взаємодіє з єдиною базою даних. Дані розподіляються між усіма компонентами програми. Навпаки, у програмі мікросервісів право власності на дані децентралізоване. Кожен сервіс є автономним і має власне приватне сховище даних, що відповідає його функціональності. Це означає, що одна служба не може змінювати дані, що зберігаються в базі даних іншої служби. І тут починаються проблеми.

Незалежно від того, яку програму ви створюєте, її мікросервіси повинні взаємодіяти та обмінюватися даними. Бо якщо цього не зробити, ви ризикуєте мати проблеми з узгодженістю, як-от дублювання даних. Проблема в тому, що в мікросервісах ви не можете використовувати транзакції ACID для транзакцій за межами однієї служби. Тож будьте обережні: наявність приватних баз даних замість однієї загальнодоступної ускладнює реалізацію запитів і транзакцій, які охоплюють декілька служб.

5 типових шаблонів, пов'язаних із даними, і 1 антишаблон

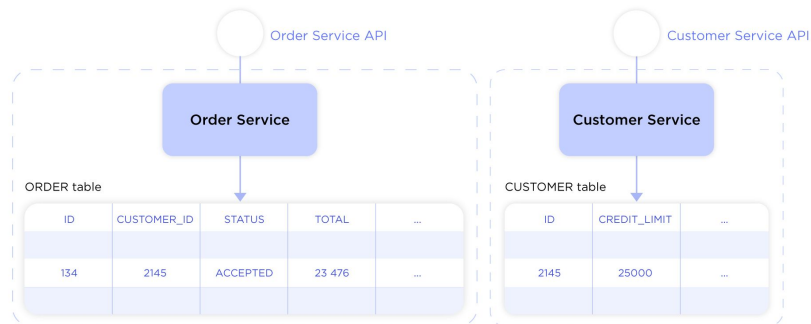
Шаблон Database-per-Service

Основною характеристикою архітектури мікросервісів є слабкий зв'язок сервісів. Для цього кожна служба повинна мати власне приватне сховище даних. Таким чином, створення архітектури бази даних для мікросервісів майже завжди вимагає дотримання шаблону database-per-service. Принаймні до моменту, коли програма ще не стала надто складною з численними службами.

Інші мікросервіси не можуть отримати прямий доступ до бази даних служби. Доступ до постійних даних кожної служби можна отримати лише через API.

Давайте розглянемо додаток інтернет-магазину. Служба замовлення та служба обслуговування клієнтів зберігають дані у власних базах даних. Зміни в одній базі даних не впливають на інші мікросервіси.

DATABASE PER SERVICE PATTERN



5 типових шаблонів, пов'язаних із даними, і 1 антишаблон

Шаблон Saga

Використання шаблону Database-per-Service також створює потребу в застосуванні шаблону Saga, оскільки sagas — це спосіб реалізації транзакцій, які охоплюють служби та підтримують узгодженість даних. У мікросервісах замість використання традиційних розподілених транзакцій (на базі ХА/2PC) потрібно використовувати послідовність локальних транзакцій (так відома як Saga). Ось як це працює: одна локальна транзакція оновлює базу даних, а потім ініціює наступну транзакцію через обмін повідомленнями.

Ви можете вибрати між двома різними підходами до координації:

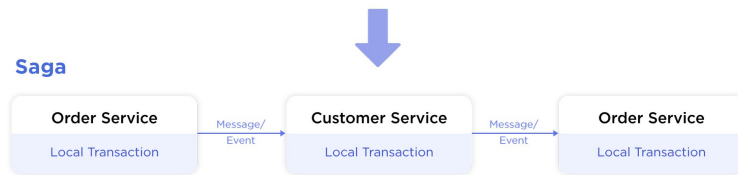
- **хореографія** – коли обмін подіями відбувається без контрольних точок
- **оркестровка** – коли у вас є централізовані контролери.

THE SAGA PATTERN

Distributed Transaction (2PC)



Saga

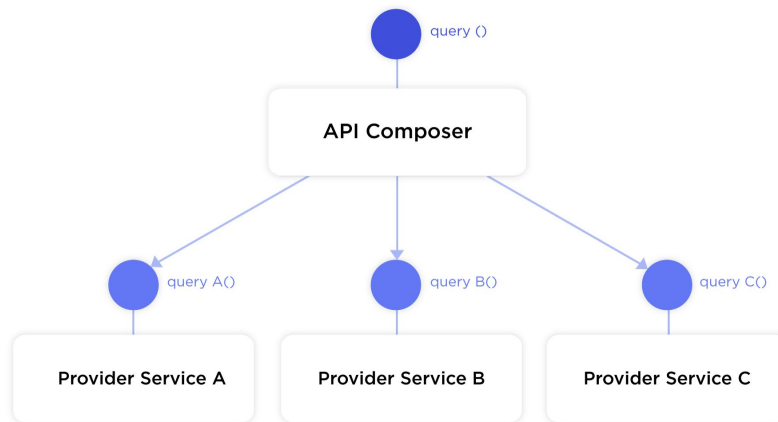


5 типових шаблонів, пов'язаних із даними, і 1 антишаблон

Шаблон композиції API

Тепер давайте детально зануримося в те, як ми можемо отримати дані, розкидані між кількома службами. Для цього ви, очевидно, не можете використовувати традиційний механізм розподілених запитів. Але ви можете замість цього використовувати шаблон Композиції API. Він реалізує запит, викликаючи певні служби, які володіють даними, і комбінуючи результати. Нижче ви можете побачити, як реалізована операція запиту. API Composer отримує дані від трьох служб постачальника.

API COMPOSITION PATTERN



5 типових шаблонів, пов'язаних із даними, і 1 антишаблон

Шаблон CQRS

Розподіл відповідальності за командний запит (CQRS) — це інший шаблон запиту, який можна використовувати. Справа в тому, що API Composition має деякі обмеження: ви не можете використовувати його для складних запитів, оскільки ви отримаєте неефективні об'єднання в пам'яті. Крім того, вам потрібно відокремити проблеми, щоб уникнути перевантаження служб, що іноді призводить до того, що служба володіє даними, не будучи тією, яка повинна реалізовувати операцію запиту.

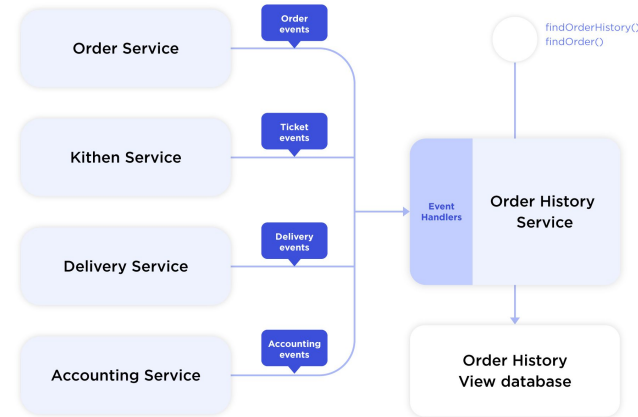
Саме тоді вам знадобиться CQRS. Він реалізує запити за допомогою баз даних перегляду. CQRS дозволяє відокремити проблеми. Він відокремлює команди від запитів. Модулі на стороні запитів реалізують запити та підтримують модель даних синхронізованою з моделлю даних на стороні команд. А що стосується модулів командної сторони, то вони керують операціями.

5 типових шаблонів, пов'язаних із даними, і 1 антишаблон

Шаблон CQRS

Ви можете застосувати CQRS у службі та використовувати її для визначення служб запитів. API служби запитів складається лише з операцій із запитами. Він запитує базу даних і підтримує її в актуальному стані, підписуючись на події, опубліковані службами, які володіють даними.

COMMAND QUERY RESPONSIBILITY SEGREGATION (CQRS) PATTERN



5 типових шаблонів, пов'язаних із даними, і 1 антишаблон

Шаблон джерела подій

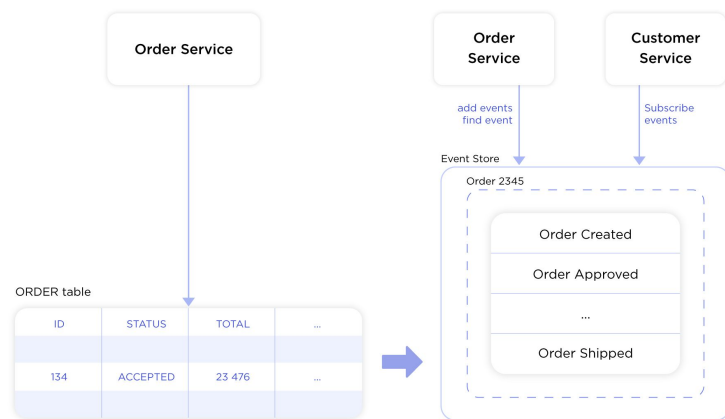
Використання Saga створює потребу в пошуку подій. Шаблон джерела подій зберігає агрегати як послідовність подій. Коли стан агрегату змінюється, нова подія зберігається в сховищі подій. Потім програма може реконструювати поточний стан об'єкта, відтворюючи події. Сховище подій використовує API для вставки та отримання подій агрегату. Він також використовує API для підписки на події, подібно до брокера повідомлень.

5 типових шаблонів, пов'язаних із даними, і 1 антишаблон

Шаблон джерела подій

Подивіться на програму, створену з джерелом подій і шаблонами CQRS. Ось як він зберігає замовлення. Кожне замовлення зберігається як послідовність подій, і клієнтський магазин підписується на них.

EVENT SOURCING PATTERN



5 типових шаблонів, пов'язаних із даними, і 1 антишаблон

Антишаблон спільної бази даних

Так, можна інтегрувати базу даних для мікросервісів. Ви можете створити єдину спільну базу даних, у якій кожна служба отримуватиме доступ до даних за допомогою локальних транзакцій ACID. Але якщо ви серйозно думаєте про це, зупиніться на цьому й двічі подумайте.

З єдиною спільною базою даних ви ризикуєте втратити всі найкращі функції мікросервісів, як-от слабкий зв'язок і незалежність сервісів. З однією базою даних вам знадобиться більше часу для координації послуг. Навіть більше, служби можуть мати проблеми під час виконання та блокувати одна одну. Крім того, якщо спільна база даних вийде з ладу, система також вийде з ладу. Тільки не забудьте зробити базу даних стійкою.

Як вибрати базу даних для мікросервісів

Сьогодні ринок баз даних пропонує безліч варіантів. Але як серед них вибрати ідеальний для свого мікросервісу? Спочатку почніть із вибору підходу до створення моделей бази даних. Давайте розглянемо два варіанти – стійкість поліглотів і багатомодельні бази даних.

Наполегливість поліглота

Архітектура мікросервісів дозволяє використовувати різні типи технологій зберігання даних для різних сервісів (така ж застосування поліглотної стійкості). Простіше кажучи, кожна команда розробників може вибрати технологію збереження, яка найкраще відповідає потребам її служби.

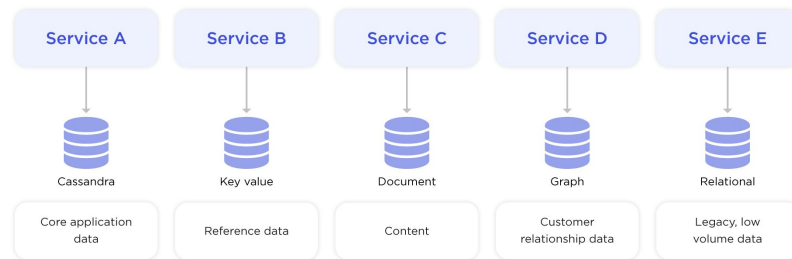
Як вибрати базу даних для мікросервісів

Наполегливість поліглота

Навіть більше, ви можете розробити службу, яка розташовується поверх кількох баз даних. Питання в тому, для чого? Ви можете мати кілька невеликих послуг замість однієї великої та складної.

RELEVANT

POLYGLOT PERSISTENCE IN MICROSERVICES



Як вибрати базу даних для мікросервісів

Багатомодельні бази даних

Крім того, ви можете використовувати багатомодельний підхід бази даних. Це дозволяє базі даних підтримувати більше однієї абстракції. Іншими словами, ви можете мати одну багатомодельну базу даних, але різні моделі даних для кожної служби, такі як ключ-значення, графіки, таблиці тощо.

Подивіться, наприклад, на корпоративну програму. У нас є програма з моделлю даних Cassandra (сховище розділених рядків) у ядрі та DSE Graph поверх неї. Існує одна база даних, але різні абстракції поверх ядра DSE для різних служб.

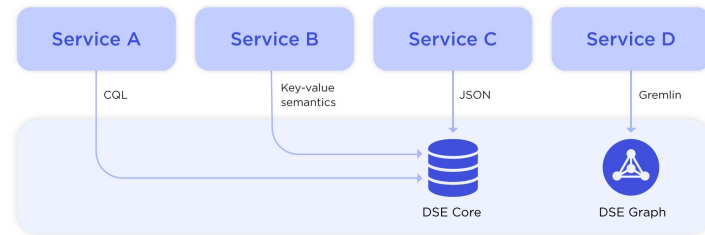
Як вибрати базу даних для мікросервісів

Багатомодельні бази даних

Мультимодельний підхід пропонує те, чого не може наполегливість поліглота – операційну простоту. Якщо у вас лише одна платформа, то легше керувати системою, навіть якщо кожен сервіс має власну модель взаємодії з даними. З іншого боку, наполегливість поліглота ідеально підходить для мікросервісів, і якщо ваша програма не надто складна, краще дотримуватися її. Хороша новина полягає в тому, що ви можете поєднати обидва підходи в кількох службах.

Тепер ви дійшли до моменту, коли вам слід вибрати технологію бази даних для мікросервісів. як? Просто запитайте себе, які дані вам потрібно зберігати та як ви плануєте підтримувати керування даними мікросервісів.

AN ENTERPRISE APPLICATION WITH A MULTI-MODEL DATABASE



Як вибрати базу даних для мікросервісів

Нереляційні бази даних

Давайте розглянемо деякі з нереляційних баз даних (NoSQL), які відповідають різним типам збережених даних і моделей даних. У мікросервісах можна об'єднати декілька з них.

Бази даних документів

Бази даних цього типу зберігають і запитують дані як JSON-подібні документи. Бази даних документів пропонують інтуїтивно зрозумілу модель даних. Іншими словами, документи відображаються на об'єкти в коді програми. Це означає, що вам не потрібно запускати JOIN або розкладати дані між таблицями. Крім того, оскільки бази даних документів є розподіленими системами, вони масштабовані.

Бази даних документів ідеально підходять для керування вмістом і зберігання каталогів. Наприклад, ви можете застосувати його в додатку для блогів або на платформі електронної комерції. Популярними базами даних документів є MongoDB і Cloudant.

Як вибрати базу даних для мікросервісів

Бази даних ключ-значення

База даних ключ-значення зберігає дані за допомогою методу ключ-значення. Це означає, що в сховищі ключ-значення дані представлені як група пар ключ-значення, де «ключ» — це те, що їх ідентифікує, а «значення» — це ідентифіковані дані. Сховище даних ключ-значення найкраще підходить для сеансово-орієнтованих програм, оскільки воно може швидко обробляти великі обсяги даних, пов'язаних із сеансом. Добре відомі бази даних ключ-значення Redis, Amazon DynamoDB, Oracle NoSQL Database.

Бази даних на основі графів

Графова база даних використовує графову модель даних, у якій сутності даних з'єднані у вузлах і формують зв'язки, що зберігаються в ребрах. Основна цінність графа полягає в зберіганні та навігації цими зв'язками. Графові бази даних зазвичай застосовуються для виявлення шахрайства, у соціальних мережах та в системах рекомендацій. OrientDB, Neo4j і Amazon Neptune — це лише кілька прикладів графових баз даних, які можна використовувати.

Як вибрати базу даних для мікросервісів

Бази даних на основі стовпців

У базі даних на основі стовпців дані зберігаються в стовпцях. Це означає, що ви можете отримати доступ до необхідних даних точніше та швидше, ніж якби вони зберігалися в рядках. Для цього ви можете просто використовувати назву стовпця, не скануючи непотрібну інформацію в рядках. Ви можете використовувати базу даних на основі стовпців для створення сховища даних. Крім того, сховище з широким стовпцем є чудовим рішенням для обробки великих даних. Перегляньте Apache Cassandra або Apache HBase, якщо вам потрібна база даних, орієнтована на стовпці.

Як вибрати базу даних для мікросервісів

Реляційні бази даних

Вам не слід розглядати використання однієї реляційної бази даних, якщо ви створюєте програму мікросервісів з нуля. Але вам доведеться працювати з ним, якщо ваша програма вже існує і вона монолітна. У такому випадку перехід на мікросервіси означатиме, що ви не зможете відразу відмовитися від старої інфраструктури. Вам доведеться почати створювати мікросервіси з того, що у вас є – реляційної бази даних, як-от DB2, MS SQL Server, MySQL, PostgreSQL, і поступово розділити її на кілька невеликих сервісів.

Крім того, ви можете використовувати реляційну базу даних у мікросервісах, якщо застосувати поліглотну стійкість. Реляційна база даних чудово підходить для служби, яка керує малими обсягами стабільних даних.

Підсумок

Якщо ви хочете мати надійну систему, вам потрібно подбати про керування даними мікросервісів. Почніть із вибору шаблонів, пов'язаних із даними. Далі слід використовувати різні підходи до створення сховищ даних і моделей. Нарешті, ви можете вибрати базу даних мікросервісів, яка найкраще відповідатиме потребам кожної служби.

Питання та відповіді